Práctica 1. Introdución a R

Estatística

Grao en Enxeñaría Informática

Profesores: Alejandro Saavedra e Ángel M. González

Índice

1. Introdución a R **3**

2. Obxectos y operacións básicas **4**

2.1. Primeiros pasos en R. Asignación de valores 4

2.2. R como *calculadora científica* 4

2.3. Tipos de variables 5

2.4. Vectores 6

2.4.1. Construción de vectores 6

2.4.2. *Lenght*, *sort* e *c* 7

2.4.3. Vectores lóxicos, caracteres e factores 7

2.4.4. Selección de compoñentes 9

2.4.5. Operacións dentro dun vector 9

2.4.6. Operacións con vectores 10

2.4.7. Produto escalar 11

2.5. Matrices 11

2.5.1. Construción de matrices 11

2.5.2. *Rownames* e *colnames* 12

2.5.3. Comandos para matrices 12

2.5.4. Operacións con matrices . . . 13
2.6. Factores . . . 14
2.7. Listas e data frames . . . 15
2.7.1. Listas . . . 15
2.7.2. Data frames . . . 15

3. Gráficos **17**
3.1. Gráficos de alto nivel . . . 17
3.1.1. A función `plot` . . . 17
3.1.2. Outros comandos gráficos . . . 18
3.2. Gráficos de baixo nivel . . . 19

4. Programación **20**
4.1. Condicións. O comando *if* . . . 20
4.2. Repeticións. Bucles *for*, *while* e *repeat* . . . 21
4.3. Funcions . . . 22

5. Librarías **23**

6. Importación de datos **24**
6.1. A función *read.table()* . . . 24
6.2. A función *read.csv()* . . . 24
6.3. Acceso ás bases de datos de R . . . 25

1. Introdución a R

R é un conxunto de ferramentas de software deseñado para a manipulación de datos, o cálculo matemático-estatístico e o procesamento de gráficos. Toda a información sobre R pódese atopar tamén na páxina web `http://www.r-project.org/`, así como as opcións para a súa descarga (ver `http://cran.es.r-project.org/`, *mirror* CRAN para España) e varios manuais para o seu uso. Algunhas das principais características de R son:

- Eficacia no manexo e almacenamento de datos.
- Contorno gráfico sinxelo e potente.
- Ferramentas de entrada/saída (lectura/escritura en ficheiros externos).
- Linguaxe de programación sinxela e ben estruturada (derivada da linguaxe C), con execución de comandos en liña: compilación e execución combinadas nun só paso.
- Uso de expresións condicionais e bucles. Desenvolvemento por parte do usuario de funcións propias.
- Software gratuíto e de código aberto (S-Plus é o seu equivalente comercial).

Ademais, R crece co traballo dos seus propios usuarios. Estes traballos gárdanse en bibliotecas (`packages`), que reúnen funcións orientadas a un determinado obxectivo: desde aplicacións de diferentes métodos estatísticos, ata a resolución de sudokus. Estas bibliotecas pódense descargar e instalar a través do propio programa (desde o menú *Paquetes*).

Para obter máis información, pode usar a axuda de R que se instala co programa (desde o menú *Ayuda*), ou calquera das aos manuais dispoñibles no seu sitio web (consulte *Manuals* na sección *Documentation* en `http://www.r-project.org/`). Tamén dispoñible: Febrero Bande, M. et al. (2008). *Estadística: ingeniería técnica en informática de sistemas*. USC (ver bibliografía básica da materia).

2. Obxectos y operacións básicas

2.1. Primeiros pasos en R. Asignación de valores

Ao executar R ábrese unha consola na que se poden escribir directamente os comandos a executar. Outra opción é crear un `script` (un arquivo de texto) onde escribir e gardar os comandos e funcións que nos interesen. Despois pódense executar copiándoos na consola.

A operación máis básica de R é asignar valores a diferentes variables. O operador que nos permitirá realizar estas asignacións é `<-`, que asigna á variable que poñemos á súa esquerda o valor que está á súa dereita.

```
>a<-1; b<-'laranxa'; c<-pi; d<-c
>a; b; c; d
[1] 1
[1] 'laranxa'
[1] 3.141593
[1] 3.141593
```

No exemplo, usamos o punto e coma (;) para poñer todos os comandos na mesma liña. Para ver que variables están almacenadas na área de traballo, úsase o comando `ls()`. Para eliminar unha variable existente, úsase `rm(nome_variable)`.

2.2. R como *calculadora científica*

Con R pódense realizar todas as operacións de cálculo elementais. Os operadores empregados son moi similares aos doutras linguaxes de programación: suma ($+$), resta ($-$), produto ($*$), división ($/$) e exponenciación (`^`). Se aparecen varios operadores binarios na mesma secuencia, a *orde priorizada* de execución é a exponenciación, despois o produto/división e por último a suma/resta. Para modificalo, temos que recorrer ao uso de parénteses.

```
>2+4*5-10/2^2
[1] 19.5
>(2+4)*5-(10/2)^2
[1] 5
```

Repárese en que para a separación decimal, R usa o punto. Este feito pode levar a problemas, por exemplo, ao importar datos de Excel onde se usa a coma para separar a parte enteira do decimal.

Ademais das operacións elementais, pódense realizar outros tipos de operacións como logaritmos (`log`), exponenciais (`exp`), raíces cadradas (`sqrt`) ou funcións trigonométricas (`sin`, `cos`, `tan`,...). Para estes casos débese aplicar a función correspondente ao valor co que opera.

```
>exp(2); sqrt(144); cos(pi/2)
[1] 7.389056
[1] 12
[1] 6.123032e-17
```

En R os valores infinitos máis e menos están representados por `Inf` e `-Inf`. Se realizamos operacións para as que as matemáticas non teñen solución, o resultado é un `NaN` (*Not a Number*).

```
>1/0;-2*Inf;sqrt(-2);0/0
[1] Inf
[1] -Inf
[1] NaN
Warning message:
In sqrt(-2) : Se han producido NaNs
[1] NaN
```

2.3. Tipos de variables

Nos exemplos que demos ata agora, todas as variables foron numéricas. Non obstante, en R podemos traballar con outros tipos de variables, como `numeric`, `logical`, `character`, `list`, `matrix`, `array`, textttfactor ou `data.frame`. Para coñecer o tipo ou clase dunha variable, utilízase a función `class`.

```
>a<-2; b<-'variable'; c<-TRUE
>class(a); class(b); class(c)
[1] 'numeric'
[1] 'character'
[1] 'logical'
```

Debemos ter en conta que un mesmo operador/función pode ter efectos diferentes segundo o tipo de obxecto ao que se aplique. Por exemplo, se facemos `2*c`, a variable lóxica primeiro codificarase como TRUE=1/FALSE=0 e despois operarase coma se fose unha variable numérica.

```
>2*c; class(2*c)
[1] 2
```

```
[1] 'numeric'
```

2.4. Vectores

opera con estructuras de datos. A máis simple é o vector, que é unha colección ordenada de elementos do mesmo tipo.

2.4.1. Construción de vectores

En dispoñemos de varios mecanismos para construír vectores.

- A forma máis común é usar o comando de concatenación `c`.

```
>x<-c(3.4,23.1,5.9,6.7,7); x
[1] 3.4 23.1 5.9 6.7 7.0
```

- Se desexamos crear un vector sen lle asignar valor algún aos seus elementos, podemos usar tamén `numeric` ou `vector`, especificando en ambos os casos a lonxitude do vector.

```
>y<-numeric(12); z<-vector(mode='numeric',length=5); y; z
[1] 0 0 0 0 0 0 0 0 0 0 0 0
[1] 0 0 0 0 0
```

- Para xerar secuencias de número enteiros podemos facer o que segue.

```
>1:5; -3:3
[1] 1 2 3 4 5
[1] -3 -2 -1 0 1 2 3
```

- Tamén podemos definir secuencias equiespaciadas nun intervalo de forma sinxela mediante a función `seq`. Temos que indicar os extremos do intervalo e, ou ben a lonxitude desexada da secuencia, ou ben a separación números consecutivos da secuencia.

```
>seq(-5,5,length=5); seq(-5,5,by=2)
[1] -5.0 -2.5 0.0 2.5 5.0
[1] -5 -3 -1 1 3 5
```

- Se o que queremos é crear un vector onde todos os seus elementos son iguais, usaremos `rep` indicando a lonxitude do vector.

```
>rep(9,4)
[1] 9 9 9 9
```

En calquera dos casos anteriores, para saber que parámetros e opcións ten un comando, podemos usar `args(nome_comando)`. Se necesitamos máis información sobre o seu uso, podemos recorrer á axuda dispoñible en R a través de `help(nome_comando)` ou `?nome_comando`. Como exemplo, podemos ver que argumentos ten a función `numeric` usando `args(numeric)` ou acceder á axuda para `seq` usando `help(seq)` ou `?seq` .

2.4.2. *Lenght*, *sort* e *c*

Para saber a lonxitude dun vector, usamos a función `length`.

```
>length(x)
[1] 5
```

Se o que nos interesa é ordenar de forma crecente ou decrecente os valores que forman o vector, podemos utilizar a función `sort`.

```
>sort(x); sort(x,decreasing=TRUE)
[1] 3.4 5.9 6.7 7.0 23.1
[1] 23.1 7.0 6.7 5.9 3.4
```

Para engadir máis valores a un vector xa existente, podemos usar de novo o comando `c`.

```
>x<-c(x,a); x
[1] 3.4 23.1 5.9 6.7 7.0 2.0
```

2.4.3. Vectores lóxicos, caracteres e factores

Ademais dos numéricos, os vectores poden ser lóxicos, de tipo carácter ou de tipo factor dependendo do tipo de variables que os formen (TRUE/FALSE, caracteres ou variables categóricas/nominais, respectivamente).

Vectores lóxicos. Os vectores lóxicos xéranse normalmente comprobando unha condición, por exemplo, comprobando se os compoñentes dun vector son negativos ou non iguais a 7.5.

```
>x<-c(4,1.3,-2.2,7.5,18,-3)
>x<0
[1] FALSE FALSE TRUE FALSE FALSE TRUE
>which(x<0)
[1] 3 6
>x!=7.5
[1] TRUE TRUE TRUE FALSE TRUE TRUE
```

No exemplo, o comando `which` devolve a posición dos elementos do vector que verifican a condición lóxica indicada (ser menores que cero, neste caso).

Vectores con caracteres. Un vector de tipo carácter permite almacenar calquera tipo de variable con resposta non sexa numérica. Os seus compoñentes estarán delimitados por aspas. Por exemplo, a resposta á pregunta "Cal é o teu equipo favorito?"

```
>y<-c('RM','FCB','RM','NIN','CEL','DEP','DEP','FCB','RM','RM','NIN')
>class(y)
[1] 'character'
>table(y)
CEL DEP FCB NIN RM
  1   2   2   2   4
```

Neste caso, a función `table` úsase para resumir a información do vector. Se algunha das persoas preguntadas non responde á pregunta, atopámonos cun valor perdido. Estes están representados en R con NA (*Not Avaliable*).

```
>y[6]<-NA
>y
[1] 'RM' 'FCB' 'RM' 'NIN' 'CEL' NA 'DEP' 'FCB' 'RM' 'RM' 'NIN'
```

A función `paste` é moi útil para concatenar secuencias de caracteres.

```
>paste("x",1:10,sep="")
'x1' 'x2' 'x3' 'x4' 'x5' 'x6' 'x7' 'x8' 'x9' 'x10'
>paste('limon','laranxa',sep='-')
'limon-laranxa'
```

Factores. Unha variable como os equipos favoritos tamén se pode declarar como `factor` dependendo das necesidades. As variables `factor` permiten un manexo cómodo de variables categóricas ou agrupadas. O comando `as.factor` permítenos converter unha variable de calquera outro tipo nun factor, mentres que `levels` permítenos coñecer as categorías da variable.

```
>y<-as.factor(y)
>y
[1] RM FCB RM NIN CEL <NA>DEP FCB RM RM NIN
Levels: CEL DEP FCB NIN RM
>class(y)
'factor'
```

```
>levels(y)
[1] 'CEL' 'DEP' 'FCB' 'NIN' 'RM'
```

2.4.4. Selección de compoñentes

Pódense seleccionar diferentes subconxuntos de elementos dun vector incluíndo entre corchetes e despois do nome do vector un *vector índice*. Un *vector índice* pode ser ou expresión lóxica ou unha secuencia numérica que indique a posición dos elementos a tomar.

```
>x[x<5&x>1]
[1] 4.0 1.3
>x[x<5|x>1]
[1] 4.0 1.3 -2.2 7.5 18.0 -3.0
>y[is.na(y)]
[1] <NA>
Levels: CEL DEP FCB NIN RM
>y[5:8]
[1] CEL <NA>DEP FCB
Levels: CEL DEP FCB NIN RM
```

Usando estes vectores índice podemos reasignar os valores dun vector naquelas posicións que nos interesen. Por exemplo, asignándolle un valor determinado as posicións que presentan valores faltantes (`NA`).

```
>x[x<5&x>1]<-3
>x
[1] -3.0 -3.0 -2.2 7.5 18 -3.0
>z<-c(8.5,9,3,5,NA,6,6.5,1,NA,4.5)
>z[is.na(z)]<-1
>z
[1] 8.5 9.0 3.0 5.0 -1.0 6.0 6.5 1.0 -1.0 4.5
```

2.4.5. Operacións dentro dun vector

Se temos un vector numérico podemos realizar numerosas operacións sobre el: a suma dos seus compoñentes (`sum`), o producto (`prod`), a media (`mean`),...

```
>x<-c(2,3,4,5)
>sum(x)
[1] 14
>prod(x)
[1] 120
>mean(x)
[1] 3.5
>sum(x)/length(x)
[1] 3.5
>sum((x-mean(x))^2)
[1] 5
```

2.4.6. Operacións con vectores

En xeral, as mesmas operacións que se poden realizar con números pódense realizar con vectores.

Operacións compoñente a compoñente. En R podemos realizar as operacións básicas sobre vectores, compoñente a compoñente.

```
>x<-c(1,5,7,11);z<-c(1,-2,3,-4)
>x+z
[1] 2 3 10 7
>x*z
[1] 1 -10 21 -44
>x/z
[1] 1.000000 -2.500000 2.333333 -2.750000
>(2*x-z)^2
[1] 1 144 121 676
```

A diferenza do que ocorre con outras linguaxes de programación, hai que ter moito coidado de que as lonxitudes dos vectores coincidan, se non, R non devolve un erro (aínda que devolve unha mensaxe de aviso) e seguirá funcionando coma se nada acontecese.

```
>t<-c(20,40,60)
>x+t
[1] 21 45 67 31
Warning message:
In x + t : longer object length is not a multiple of shorter object length
```

E vemos que no cuarto compoñente do vector suma reciclou o primeiro compoñente do vector t, realizando a operación $20+11=31$. Do mesmo xeito, pódense realizar operacións vectoriais sobre números. Neste caso, a operación aplícase a cada compoñente do vector.

```
>x^2
[1] 1 25 49 121
>seq(-5,5,length=20)+5
[1] 0.0000000 0.5263158 1.0526316 1.5789474 2.1052632 2.6315789
[7] 3.1578947 3.6842105 4.2105263 4.7368421 5.2631579 5.7894737
[13] 6.3157895 6.8421053 7.3684211 7.8947368 8.4210526 8.9473684
[19] 9.4736842 10.0000000
```

2.4.7. Produto escalar

O produto escalar de dous vectores, como se coñece en matemáticas $\sum_i x_i z_i$, realízase co operador %*%.

```
>x%*%z
[1,] -32
```

2.5. Matrices

2.5.1. Construción de matrices

As matrices son xeneralizacións bidimensionais de vectores. O comando `matrix` é usado habitualmente para crear unha matriz. Nos parámetros deste comando é necesario indicar os compoñentes da matriz e o número de filas e columnas. Outros comandos útiles para construír matrices son `rbind` e `cbind` que unen vectores por filas ou por columnas para construír unha matriz.

```
>A<-matrix(c(1,2,3,7),nrow=2,ncol=2)
>A
[1,] 1 3
[2,] 2 7
>B<-matrix(1:6,nrow=2,ncol=3)
>B
[1,] 1 3 5
[2,] 2 4 6
>A<-cbind(c(1,2),c(3,7))
>B<-rbind(c(1,3,5),c(2,4,6))
```

```
>A;B
```

2.5.2. *Rownames* e *colnames*

Nas bases de datos coas que adoitamos traballar tanto en Estatística como noutras ciencias experimentais, as filas representan xeralmente individuos e as columnas as variables a estudar. Como consecuencia, pode ocorrer que queiramos darlle nome a determinadas filas ou columnas dunha matriz. Isto faise en R mediante os comandos `colnames` e `rownames`.

```
>colnames(A)
NULL
>colnames(A)<-c('Variable1','Variable2')
>A
Variable1 Variable2
[1,] 1 3
[2,] 2 7
>colnames(A)<-paste('Variable',1:2,sep=)
>A
Variable1 Variable2
[1,] 1 3
[2,] 2 7
```

2.5.3. Comandos para matrices

Hai moitos comandos deseñados para traballar con matrices: `diag` para extraer a diagonal, `det` para o determinante, `solve` que se usa para obter a inversa (sempre que a matriz sexa non singular), ...

```
>diag(A)
[1] 1 7
>t(A)
Variable1 1 2
Variable2 3 7
>det(A)
[1] 1
>solve(A)
Variable1 7 -3
Variable2 -2 1
>A%*%solve(A)
```

```
[1,] 1 0
[2,] 0 1
>solve(A)%*%A
Variable1 1 0
Variable2 0 1
```

Ademais, `eigen` permítenos obter os valores propios e os vectores propios asociados a unha matriz.

```
>eigen(A)
$values
[1] 7.8729833 0.1270167
$vectors
[1,] -0.4000430 -0.9601733
[2,] -0.9164964 0.2794051
```

2.5.4. Operacións con matrices

Operacións compoñente a compoñente. Tamén se poden usar operadores de cálculo comúns para operar compoñente por compoñente con matrices. Aquí, ao contrario do que ocorre cos vectores, os desaxustes dimensionais darán lugar a erros.

```
>A+B
Error en A + B : arreglos de dimensión no compatibles
>C<-matrix(c(5,3,7,8),nrow=2,ncol=2)
>A+C
[1,] 6 10
[2,] 5 15
>A*C
[1,] 5 21
[2,] 6 56
```

Operacións *matriciais*. A partir dun produto "matricial"de vectores (xa que un vector pódese pensar como unha matriz cunha soa fila ou columna) tamén se pode xerar unha matriz. O comando `t` úsase para obter a transposición dunha matriz (ou dun vector).

```
>x<-c(1,2,3);y<-c(4,5,6)
>t(x)
[1,] 1 2 3
>x%*%t(x)
```

```
[1,] 1 2 3
[2,] 2 4 6
[3,] 3 6 9
>x%*%t(y)
[1,] 4 5 6
[2,] 8 10 12
[3,] 12 15 18
```

O produto de matrices comúnmente coñecido en matemáticas realízase como o produto vectorial que acabamos de ver mediante o operador %*%.

```
>A%*%B
[1,] 7 15 23
[2,] 16 34 52
```

2.6. Factores

Como anticipamos brevemente nunha subsección anterior, `factor` é a clase xeralmente asignada en R ás variables categóricas. O comando `factor` é o que proporciona R para xerar variables deste tipo. No seguinte exemplo xeramos unha variable categórica onde os seus compoñentes son as letras da palabra 'estatística' e os niveis son as letras do alfabeto (sen o ñ).

```
>letters
[1] 'a' 'b' 'c' 'd' 'e' 'f' 'g' 'h' 'i' 'j' 'k' 'l' 'm' 'n' 'o' 'p' 'q' 'r'
[20] 's' 't' 'u' 'v' 'w' 'x' 'y' 'z'
>substring("estatistica",1:11,1:11)
[1] 'e' 's' 't' 'a' 't' 'i' 's' 't' 'i' 'c' 'a'
>factor(substring('estatistica',1:11,1:11),levels=letters)
[1] e s t a t i s t i c a
Levels: a b c d e f g h i j k l m n o p q r s t u v w x y z
```

A maiores, as variables de factores poden estar formadas tanto por caracteres como por números.

```
>factor(sample(1:10,size=40,replace=TRUE),levels=1:10)
[1] 3 7 10 6 3 3 5 4 4 8 4 6 3 1 10 7 4 1 9 4 10 2 8 7 8
[26] 2 3 2 5 4 8 10 6 2 2 1 2 2 2 9
Levels: 1 2 3 4 5 6 7 8 9 10
```

O comando `levels` aplicado a unha variable factor permítenos coñecer os seus niveis.

```
>levels(factor(sample(1:10,size=40,replace=TRUE),levels=1:10))
[1] '1' '2' '3' '4' '5' '6' '7' '8' '9' '10'
```

2.7. Listas e data frames

2.7.1. Listas

Unha lista en é un obxecto que consiste nunha colección ordenada doutros obxectos coñecidos como compoñentes da lista. Ditos compoñentes poden ser de diferentes clases, dimensións, etc. O comando utilizado para xerar unha lista é `list` e os seus parámetros, os compoñentes da devandita lista xunto con cadanseu nome.

```
 >Lista<-list(marido="Juan Pedro",esposa="Josefina",nfillos=3,
> + idadefillos=c(16,11,4),satisfaccion=matrix(c(8,7,5,4,4,3,5,10,7,7),
> + nrow=5,ncol=2))
```

Hai dúas formas de acceder individualmente aos compoñentes dunha lista. O primeiro é a través do número de compoñente, para o que temos que utilizar o corchete dobre despois do nome da lista. O segundo é a través do nome do compoñente, escribíndoo despois do nome da lista precedido do carácter dólar $.

```
>Lista[[2]]
[1] 'Josefina'
>Lista[[5]]
[1,] 8 3
[2,] 7 5
[3,] 5 10
[4,] 4 7
[5,] 4 7
>Lista$marido
[1] 'Juan Pedro'
>Lista$idadefillos
[1] 16 11 4
```

2.7.2. Data frames

Outra clase de obxectos cos que teremos que afacernos a traballar son os data frames. Un marco de datos é unha lista cuxos compoñentes deben cumprir unha serie de requisitos. O comando xeral para a construción de data frames é `data.frame`.

```
>cbind(1,1:5)
[1,] 1 1
[2,] 1 2
```

```
[3,] 1 3
[4,] 1 4
[5,] 1 5
>L3 <- LETTERS[1:3]
>L3
[1] 'A' 'B' 'C'
>data.frame(cbind(1,1:5),sample(L3,5,replace=TRUE))
X1 X2 sample.L3..5..replace...TRUE.
1 1 1 C
2 1 2 A
3 1 3 A
4 1 4 A
5 1 5 C
```

É habitual que, debido aos requisitos de determinadas funcións, ás veces nos vexamos obrigados a converter matrices en data frames. De xeito análogo ao que pasou cos factores, a orde de execución para isto é `as.data.frame`.

```
>matriz4x4<-matrix(1:16,nrow=4,ncol=4)
>as.data.frame(matriz4x4)
V1 V2 V3 V4
1 1 5 9 13
2 2 6 10 14
3 3 7 11 15
4 4 8 12 16
```

Aínda que o entorno que nos ofrece o programa R non parece atractivo como o doutros programas comerciais, como S-Plus ou SPSS, R tamén nos ofrece a posibilidade de visualizar os datos da matriz ou os data frames como unha táboa. Isto conséguese mediante o comando `fix`.

```
>fix(matriz4x4)
```

Para poder seguir usando a consola de comandos despois de usar `fix` será necesario pechar a xanela que se abre ao usalo.

3. Gráficos

As gráficas representan un compoñente moi importante en R. Mediante gráficas é posible representar un gran número de funcións estatísticas. Os comandos gráficos en R divídense basicamente en 3 grupos:

- **Gráficos de alto nivel.** Crea un novo gráfico na pantalla; opcionalmente inclúe etiquetas, títulos, lendas, ...
- **Gráficas de baixo nivel.** Engade información aos gráficos existentes. Por exemplo, puntos, liñas, ...
- **Gráficos interactivos.** Permítelle ao usuario engadir ou eliminar información dun gráfico de forma interactiva, así como utilizar o rato como punteiro ou localizador.

Todos os comandos e funcións gráficas que veremos aquí están incluídos nos paquetes que están instalados por defecto en R. Hai paquetes gráficos opcionais (e tamén gratuítos) que se poden instalar en R, como *maps*, *grid* ou *lattice*, que proporcionan ao usuario funcionalidades máis avanzadas.

3.1. Gráficos de alto nivel

Os gráficos de alto nivel están deseñados para xerar un gráfico completo a partir dos datos que se pasan como argumentos á función. Os eixos, as etiquetas e os títulos xéranse automaticamente a non ser que o usuario requira o contrario. Os gráficos de nivel superior sempre inician un novo gráfico, eliminando o actual se non se indica o contrario. Se queremos abrir unha nova pantalla gráfica sen borrar a anterior, debemos utilizar o comando `windows()`.

3.1.1. A función `plot`

Sen dúbida a función gráfica máis usada é `plot`. É unha función xenérica, onde a gráfica producida depende da clase do primeiro argumento da función. Vexamos un exemplo do seu uso, onde se xera unha gráfica de dispersión de peso fronte a estatura nun grupo de 10 individuos.

```
>Altura<-c(175,177,192,159,166,188,180,182,177,163)
>Peso<-c(77,70,101,51,55,87,78,99,74,50)
>plot(Altura,Peso)
```

Se o primeiro dato que se pasa á función `plot` é un `factor` (por exemplo, o xénero de cada un dos individuos), a gráfica que se xera é unha gráfica de caixas e bigotes (boxplot) dos segundos datos (`Altura` ou `Peso`) para cada nivel do factor `Sexo`. Vexamos a continuación como se xerarían dous diagramas de caixas e bigotes para cada nivel de `Sexo` para `Altura` e `Peso` respectivamente.

```
>Sexo<-as.factor(c('H','M','H','M','M','H','H','H','M','M'))
>plot(Sexo,Altura,col=3)
>windows()
>plot(Sexo,Peso,col=4)
```

Usamos o parámetro `col`, para indicarlle a R a cor na que queremos o gráfico. Isto pódese facer numericamente ou indicando directamente a cor cun comando `col=''blue''`. Pódese atopar un catálogo das cores dispoñibles en R en: `https://r-charts.com/colors/`.

Ás veces podemos estar interesados en xerar unha única pantalla gráfica que conteña multitude de gráficos diferentes. Supoñamos, por exemplo, que na mesma pantalla pretendemos incluír os tres gráficos xerados anteriormente e tamén un gráfico que represente a evolución da altura media na última década (unha serie temporal).

```
>par(mfrow=c(2,2))
>plot(Altura,Peso)
>plot(Sexo,Altura,col=3)
>plot(Sexo,Peso,col=4)
>EstatMedia<-c(166.8,168.2,168.1,168.9,170.5,170.2,170.6,172,173.1,174.2)
>plot(EstatMedia,type='l',lwd=2,col='cyan',lty=3)
```

Aquí, co comando `par(mfrow=c(2,2))` dicímoslle a R que divida a pantalla gráfica en 2 filas e 2 columnas, para que poidamos incluír $2 \times 2 = 4$ gráficos distintos nela. Ademais, na última das gráficas, estamos a utilizar parámetros que descoñecíamos ata agora. `type='l'` utilízase para indicar que queremos que os puntos da gráfica estean unidos a través de liñas, outras opcións son `type= 'p'` (para puntos) ou `type='b'` (para puntos e liñas). O parámetro `lty` úsase para indicar o tipo de liña desexada. Por exemplo, `lty=1` é a liña continua habitual e `lty=3` é a liña discontinua. Co parámetro `lwd` indicamos o ancho de liña desexado.

3.1.2. Outros comandos gráficos

Aínda que a función `plot` é a máis importante e a máis usada, hai outras funcións que permiten o trazado de alto nivel.

- `pairs` debuxa gráficos de dispersión de 2 por 2 entre todas as columnas que forman un data

frame ou unha matriz numérica.

- `hist` debuxa o histograma dunha variable.
- `image` crea un rectángulo a partir dunha paleta de cores onde a cor cambia en función dos datos do terceiro argumento.
- `contour` crea un diagrama de contorno baseado nos valores do terceiro argumento.
- `persp` é o equivalente en 3D do diagrama de dispersión 2D que debuxamos anteriormente con `plot`.

A continuación móstranse algúns exemplos breves de uso destes comandos.

```
>X<-matrix(sample(1:100,size=50*5,replace=TRUE),nrow=50,ncol=5)
>pairs(X)
>windows()
>hist(Peso)
>windows()
>x <- 10*(1:nrow(volcano))
>y <- 10*(1:ncol(volcano))
>image(x, y, volcano, col = terrain.colors(100),main = 'Volcan',
> + sub = 'Hawai, EEUU')
>windows()
>contour(x,y,volcano,col=2:3)
>windows()
>persp(x,y,volcano,theta=30,phi=40)
```

No uso que aquí lle damos á función `image` podemos atopar un novo exemplo de parámetro que se pode aplicar a gráficos de alto nivel: `main`. Utilízase para darlle un título ao gráfico, mentres que dun xeito análogo pódese aplicar un subtítulo mediante `sub`. `theta` e `phi` son parámetros de `persp` que nos permiten cambiar o punto de vista desde o que miramos a gráfica.

3.2. Gráficos de baixo nivel

Ás veces, as funcións que crean gráficos de alto nivel non producen exactamente o tipo de gráfico que quere o usuario. Neste caso, pódense usar comandos gráficos de baixo nivel para engadir información adicional ao gráfico actual. No seguinte exemplo mostramos algúns dos gráficos de baixo nivel máis comúns. A función `points` permite engadir un ou máis puntos a un gráfico especificando as súas coordenadas, o parámetro `pch` é usado polo o usuario para indicar como quere marcar tales puntos (círculo sólido ou oco, rombo, triángulo, ...). `line` permítelle engadir segmentos de liña a

un gráfico preexistente, indicando as coordenadas dos puntos a unir. O comando `abline` permítelle incluír rectas dentro dun gráfico, indicando o seu termo independente e a súa pendente.

```
>plot(Altura,Peso)
>points(c(165,184),c(80,72),col='green',pch=2)
>lines(c(159,192),c(51,101),lwd=2.5,col=29)
>abline(a=72,b=0,col="red",lty=2,lwd=3)
```

Ademais de `points`, `lines` e `abline`, hai moitos gráficos de baixo nivel en R. Aquí só nos referiremos a dous máis: `legend`, que permite incluír unha lenda nun gráfico preexistente e `title`, que lle permite engadir títulos e subtítulos a un gráfico xa creado, sempre que non se fixera antes.

```
>x<-seq(-pi,pi,length=65)
>plot(x,sin(x),type='l',ylim=c(-1.2,1.8),col= 3,lty=2)
>points(x,cos(x),pch=3,col=4)
>lines(x,tan(x),type='b',lty=1,pch=4,col=6)
>legend(-1,1.9,c('sin','cos','tan'),col=c(3,4,6),lty=c(2,-1,1),pch=c(-1,3,4))
>title(main='Funciones trigonométricas',sub='seno, coseno e tangente')
```

4. Programación

R é unha linguaxe de expresión no sentido de que os seus comandos, funcións ou expresións devolven un resultado ao usuario. Como vimos, un script pódese agrupar, ben separado por saltos de liña ou punto e coma. Non obstante, toda linguaxe de programación, para a resolución de problemas complexos, debe contar con comandos que permitan unha execución controlada a partir de expresións lóxicas, como condicionais e repeticións, que estudaremos a continuación.

4.1. Condicións. O comando *if*

As instrucións condicionais execútanse mediante o comando `if` e, como o seu nome indica, executan unha determinada instrución (que vai entre chaves) sempre que unha determinada expresión lóxica (que vai entre parénteses xusto despois do `if`). Opcionalmente, poden incluír un `else` e inmediatamente despois unha instrución entre chaves que se executará sempre que non se cumpra a expresión lóxica despois do `if`.

```
>l<-sample(1:10,size=1)
>if (l<6) {print('entre 1 e 5')} else {print('entre 6 e 10')}
```

Aquí estamos simulando, con igual probabilidade, un número enteiro entre 1 e 10 e, condicional-

mente se o número simulado está entre 1 e 5 ou entre 6 e 10, imprimímolo na pantalla, usando o comando `print`. Obviamente, os condicionais pódense facer máis complexos segundo as necesidades específicas do usuario, usando `else if`

```
>l<-sample(1:10,size=1);t<-sample(1:10,size=1)
>if (l<t) {print(c(l,t,l*t))} else if (l==t) {print(c(l,t,l**t))}
>else {print(c(l,t,l/t))}
```

A complexidade dos condicionais tamén está relacionada coa complexidade das expresións lóxicas que van dentro das parénteses. Supoñamos que nun exemplo como o anterior nos interesa imprimir en pantalla o produto de `l` por `t` sempre que polo menos un dos dous estea por debaixo de 4 e o produto tome un valor superior a 20. A declaración condicional sería a seguinte:

```
>l<-sample(1:10,size=1);t<-sample(1:10,size=1)
>if ((l<4|t<4)&(l*t)>20) {print(l*t)}
```

É moi habitual que, ao programar scripts, o usuario queira engadir comentarios para dar explicacións sobre os pasos que se están dando en cada momento. Isto consíguese mediante o carácter #, de xeito que R non ten en conta todo o que vai detrás del.

```
># Nesta liña non se executa nada.
>print(c(l,t)) # Isto é un comentario.
```

4.2. Repeticións. Bucles *for*, *while* e *repeat*

Moitas veces quereremos repetir unha determinada secuencia de ordes un número determinado de veces, ou ata que se produza unha determinada condición. En R hai 3 comandos posibles para a repetición de scripts.

- `for` úsase para a repetición finita de instrucións nun bucle.
- `while` repite as instruccións mentras se siga verificando unha certa expresión lóxica.
- `repeat` repite as instrucións ata chegar a `break`, que detén a execución do bucle.

A continuación móstrase un exemplo de uso do bucle `for`, onde os elementos dun vector (previamente declarado) se calculan mediante un bucle que se executa ao longo do devandito vector.

```
>vec<-numeric(20)
>for (i in 1:length(vec)) {
> +   vec[i]<-sqrt((i*3-17)/2)
```

```
> +   }
>vec
```

Para o comando `while`, temos que volver ás expresións lóxicas, igual que fixemos con `if`, debido á natureza condicional da repetición.

```
>j<-0;k<-0
>while(j<1e20|k<40) {j<-2**k;k<-k+1;print(c(j,k))}
```

Finalmente, o comando `repeat` repite unha secuencia de ordes sen condicións nin puntos de interrupción, polo que a única forma de detelo é mediante un `break`. No seguinte exemplo calcúlase un vector cuxas compoñentes son as potencias sucesivas de 2, ata que se interrompe a execución cando as potencias xa son excesivamente grandes.

```
>l<-1;s<-1
>repeat {
> +   l<-c(l,2**s);s<-s+1;if (l[length(l)]>1e8) {break}
> +   }
>l
```

4.3. Funcions

Unha das vantaxes da linguaxe de programación R , e ao mesmo tempo unha das claves do seu crecemento e desenvolvemento de paquetes e bibliotecas, é que permite ao usuario crear obxectos de tipo función. As funcións son secuencias de comandos e comandos que devolven unha ou máis saídas e poden usarse en expresións posteriores. Deste xeito, a linguaxe de programación R medra en potencia e elegancia, e o seu uso faise máis agradable e produtivo.

Como exemplo, mostramos unha función cuxos parámetros de entrada son dous vectores `y1` e `y2`, e que devolve como saída a media e a desviación estándar da subtracción destes vectores.

```
>calculo<-function(y1,y2) {
> +   Ry1y2<-y1-y2
> +   return(list('media'=mean(Ry1y2),'desv'=sd(Ry1y2)))
> +   }
```

Pódese ver que un dos comandos fundamentais dentro da función é `return`, que indica os parámetros que devolve a función. Para chamar á función e obter a saída, o procedemento sería o seguinte.

```
>x<-rnorm(50);z<-rnorm(50)
>calculo(x,z)
$media
[1] -0.06090414
$sd
[1] 1.559908
```

Na chamada da función é importante saber que os parámetros de entrada non precisan levar o mesmo nome que tiñan cando se creou a función. Unha función pode ter tantos parámetros de entrada ou saída como se queira, e poden ser de diferentes tipos e clases.

Cando un usuario de R crea funcións, é moi probable que se produzan erros, tanto na programación como nos posibles cálculos/ordes que se realicen no seu interior, especialmente cando a función é moi grande. Nestes casos, para depurar o programa, é moi útil usar a función `print` para escribir a saída na pantalla.

5. Librarías

Como mencionamos anteriormente, unha das vantaxes máis importantes de R é que cada usuario pode construír as súas propias funcións, almacenalas en bibliotecas ou paquetes (*packages*) e facelos públicos para que estean dispoñibles para outros usuarios. Para saber que paquetes xa están instalados no noso contorno R , executamos `library()`. Para que un determinado paquete funcione, non só ten que estar instalado, tamén ten que cargalo na consola R . Para saber cales dos paquetes instalados están cargados no noso entorno, executamos `search()`.

En moitas ocasións, descubrimos que o paquete que nos interesa non está instalado en R . Se tes conexión a Internet, no menú R vai a *Paquetes* e a *Instalar paquete(s)...* , onde o usuario ten que escoller o *mirror* que máis próximo lle sexa. Unha vez feito isto, aparece na pantalla unha lista (que crece co paso do tempo) dos paquetes dispoñibles. Despois de instalalo, cárgase na pantalla mediante o comando `library(nome_paquete)` e todas as súas funcións estarán dispoñibles.

Se se quere obter información sobre un paquete cargado na nosa consola, podemolo facer executando `help(name_package)`, ou descargar a axuda en pdf do paquete dispoñible en `http://cran.es.r-project.org/`.

6. Importación de datos

Aínda que R e moitos dos seus paquetes proporcionan xa aos usuarios varias bases de datos, en moitas ocasións quereremos traballar con datos almacenados en ficheiros externos, sexan ficheiros de texto, Excel, SPSS, ... Os comandos e funcións de R para importar datos son sinxelos pero tamén estritos e bastante inflexibles. Isto débese principalmente a que os deseñadores da linguaxe R cren que non é difícil para o usuario modificar os seus propios ficheiros para que cumpran os requisitos de R.

6.1. A función *read.table()*

A función *read.table()* úsase normalmente para ler datos de ficheiros de texto externos (cunha extensión .txt). O seu formato é moi sinxelo e o número de parámetros da función é moi grande, como se pode ver na súa axuda (escribindo `?read.table`).

Un exemplo da aplicación desta orde pódese ver descargando en formato texto calquera base de datos de interese que atopemos en Internet. Por exemplo, na páxina web do Instituto Galego de Estatística (IGE), `https://www.ige.eu/`, podemos atopar multitude de bases de datos que miden os tipos de interese da poboación galega. Podemos descargar, a modo de exemplo, a base de datos coas cifras de poboación das catro provincias galegas. O comando correcto para importar tales datos a R sería:

```
>DatosPob<-read.table(file='Ruta_DatosPoboacion.txt',sep='t',header=TRUE)
```

En `file` indicamos a R a ruta a seguir (no propio ordenador) para chegar ao ficheiro desexado. `header` é un parámetro lóxico para indicar a R se a primeira fila do ficheiro corresponde (TRUE) ou non (FALSE) a unha cabeceira que indica os nomes das variables. Con `sep` indicamos a R cal é a separación entre variables nun ficheiro (espazo, separador, coma, ...). Opcionalmente, pódese utilizar calquera dos outros parámetros que aparecen na axuda, como `dec`, co que indicamos a R o separador decimal empregado no ficheiro de texto inicial.

6.2. A función *read.csv()*

O comando *read.csv()* é o equivalente a *read.table()* para importar datos de ficheiros de Excel con extensión .csv a R (nalgúns casos será necesario utilizar *read.csv2()*). Na axuda de R pode-mos atopar a súa forma de uso:

```
>read.csv(file, header = TRUE, sep = ',', quote='', dec='.',
```

```
> + fill = TRUE, comment.char="", ...)
```

Un exemplo da aplicación desta orde tamén se pode ver na base de datos de poboación das capitais de provincia, que xa utilizamos anteriormente:

```
>DatosPob2<-read.csv(file='Ruta_DatosPoboacion.csv',sep=';',header=TRUE)
```

En casos coma este, é necesario que teñamos gardado o ficheiro Excel coa extensión .csv, indicando que a separación se fixo con punto e coma. Aínda que neste caso non é necesario, dado que os datos de poboación son enteiros, hai que ter moito coidado ao traballar con ficheiros Excel, xa que a separación decimal adoita ser unha coma, o que obrigaría a engadir `dec =','` a os parámetros.

6.3. Acceso ás bases de datos de R

R proporciona aproximadamente cen bases de datos, ben a través do paquete `datasets` (que se instala por defecto con R), ben a través doutros paquetes. Podemos ver a lista de bases de datos ás que temos acceso desde R usando o comando `data()`. O acceso ás bases de datos que se atopan nos paquetes base de R realízase a través do seu nome. Obviamente, antes de utilizar estas bases de datos, interesarémonos en coñecer as súas características, para o que faremos un `help` sobre o nome destas bases:

```
>CO2
>help(CO2)
```

Pode ocorrer que en determinadas ocasións esteamos interesados en ter as bases de datos presentes en paquetes R que non están instalados por defecto co programa. Se por exemplo queremos coñecer todas as bases de datos do paquete `rpart`:

```
>data(package='rpart')
```

Se, pola contra, quixeramos acceder á base de datos `imports85` dentro do paquete `randomForest`, escribiriamos:

```
>data(imports85,package='randomForest')
```